

Principal Component Analysis(PCA)

- PCA (Principal component analysis) is a dimensionality reduction technique used to reduce the number of features while retaining most of the information in the dataset.

Principal Components

- Principal components are new variables created from the original features.

PC1 (First Principal Component)

- Captures the maximum variance in the data.
- Most information component.

PC2 (Second Principle Component)

- Captures the remaining variance .
- Perpendicular to PC1

PC3 , PC4 ,....

- Capture progressively less variance.

Example:

Original features.

- height
- Age

- Income
- weight

After PCA

- PC1
- PC2

Explained Variance Ratio

- Explained Variance Ratio tells how much information each principal component retains.

Example

Component	Variance Explained
PC1	70%
PC2	20%
PC3	7%
PC4	3%

Total = 100%

Interpretation

- PC1 contains 70 % of the information.
- PC2 contains 20 % of the information.
- PC1 + PC2 contain 90% of the information.

- PC1 + PC2

and remove the remaining components.

Scree Plot

A scree plot shows : Principle Component vs Explained Variance .

Example:

```
Variance
|
| *****
| *****
| ***
| *
|
+-----+
PC1  PC2  PC3  PC4
```

Purpose of Scree plot:

- Find how many components to keep.
- Identify the elbow point.
- Components after the elbow contribute very little information.

When to Apply:

- Data visulization.
- Reduce overfitting.
- Highly Correlated feature
- Faster training needed

PCA Overflow

```

Data
|
Standardiation
|
PCA
|
Principal components
|
Train model
    
```

```

[1]: from sklearn.datasets import load_iris
      from sklearn.preprocessing import StandardScaler
      from sklearn.decomposition import PCA
      import matplotlib.pyplot as plt
      import pandas as pd

      # Load dataset
      iris = load_iris()
      X = iris.data

      # Standardize data
      scaler = StandardScaler()
      X_scaled = scaler.fit_transform(X)

      # Apply PCA
      pca = PCA(n_components=2)
      X_pca = pca.fit_transform(X_scaled)

      # Explained Variance Ratio
      print("Explained Variance Ratio:")
      print(pca.explained_variance_ratio_)
    
```

```

# PCA DataFrame
df = pd.DataFrame(
    X_pca,
    columns=["PC1", "PC2"]
)

print(df.head())

# Scatter Plot
plt.scatter(X_pca[:,0], X_pca[:,1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization")

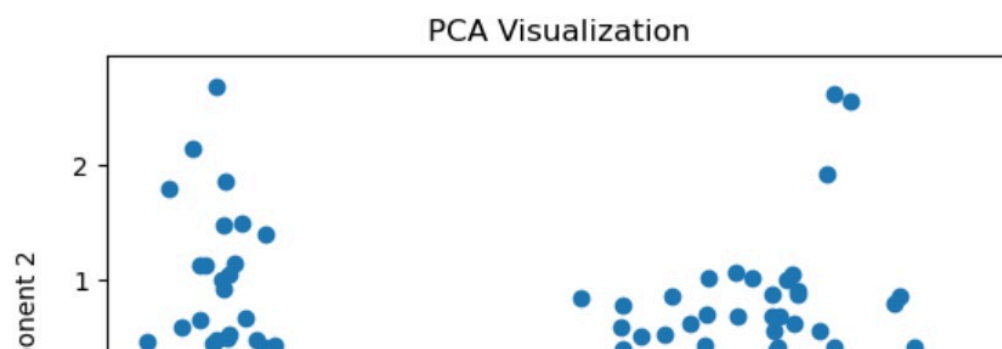
plt.show()

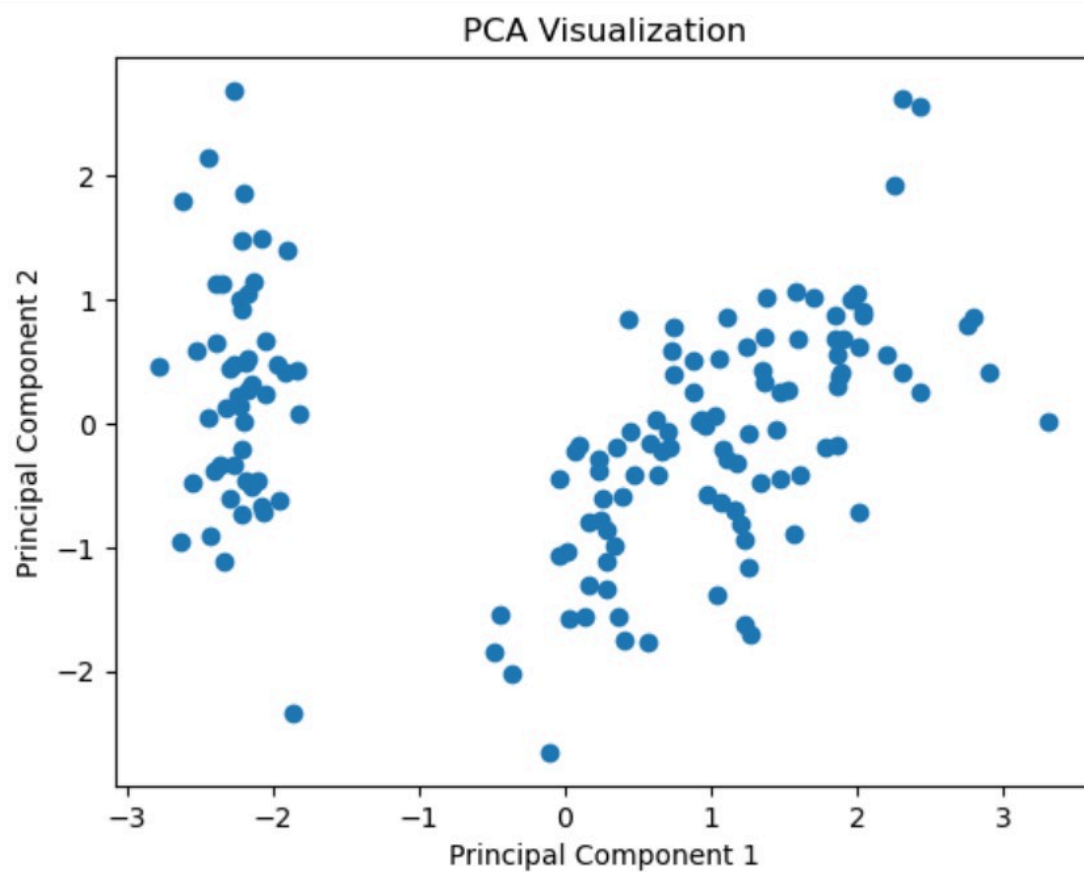
```

Explained Variance Ratio:

[0.72962445 0.22850762]

	PC1	PC2
0	-2.264703	0.480027
1	-2.080961	-0.674134
2	-2.364229	-0.341908
3	-2.299384	-0.597395
4	-2.389842	0.646835



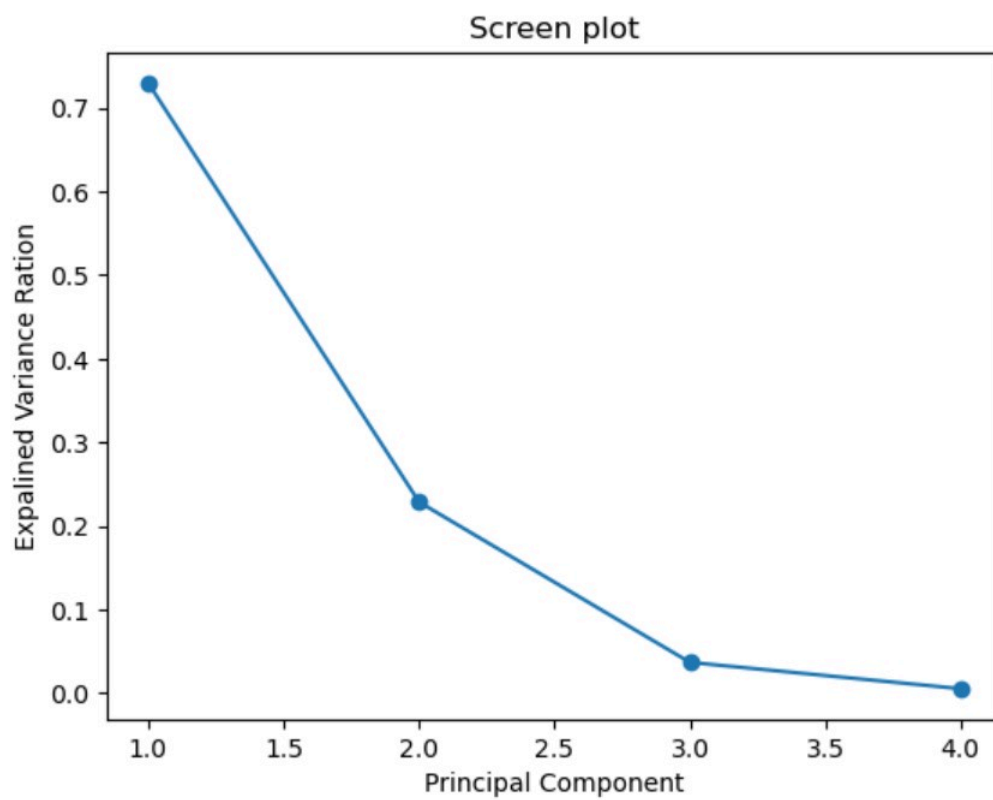


```
[2]: #Screen plot
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca=PCA()
pca.fit(X_scaled)

plt.plot(
```

```
range(1, len(pca.explained_variance_ratio_) + 1),  
pca.explained_variance_ratio_,  
marker='o')  
  
plt.xlabel("Principal Component")  
plt.ylabel("Expalined Variance Ration")  
plt.title("Screen plot")  
plt.show()
```



Cross Validation

- Cross Validation is a technique used to evaluate a machine learning model by training and testing it on different subsets of the data.
- Its main purpose is to check how well the model generalizes to unseen data.

Why is a Single Train-Test Split Insufficient .

- Suppose we split the data as :
 - Training Data = 80%
 - Testing Data = 20%

Problem :

Performance depends on how the data was split.

- A lucky split may give very high accuracy.
- An unlucky split may give low accuracy.

Results can be missing.

Example:

```
Split 1    Accuracy = 95%
Split 2
```

K-Fold Cross Validation .

In K-Fold Cross Validation , the datasets is divided into K equal parts(folds)

Example

- $k=5$
- Fold1
- Fold2
- Fold3
- Fold4
- Fold5

Process

Iteration 1

- Test $\rightarrow$ Fold1
- Train $\rightarrow$ Fold2+Fold3+Fold4+Fold5

Iteration 2

- Test $\rightarrow$ Fold2
- Train $\rightarrow$ Remaining folds.

Continue until every fold is used once as test data.

Final Score

Average of all K accuraries.

Example

- Fold 1 = 92%

- Fold 2 = 90%
- Fold 3 = 94%
- Fold 4 = 91%
- Fold 5 = 93%
- $(92+90+94+91+93)/5$
 - = 92%

Benefits :

- Used all data for training and testing.
- More reliable evaluation.
- Reduce bias from a single train test split.
- Better estimate of model performance.

Stratified K-Fold

Normal K-Fold may create folds with unequal class distributions.

Example Dataset

90 Pass 10 Fail Some folds may contain very few "Fail" samples.

This can lead to unreliable evaluation

Stratified K-Fold Solution

Example

Original dataset :

- 90 % Pass
- 10% Fail

Each fold:

- 90 % Pass
- 10% Fail

The same class ratio is maintained in every fold.

Benefits:

- More reliable evaluation.
- Better for classification problems.
- Handles imbalanced datasets.
- Maintains class proportional.

K-Fold vs Stratified K-Fold

K-Fold	Stratified K-Fold
Random split into K folds	Preserves class distribution
Suitable for balanced data	Suitable for imbalanced data
May create uneven classes	Keeps class proportions same
General-purpose	Mostly used in classification

[3]: *#K-Fold Cluster*

```

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import KFold, cross_val_score

```

```

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

```

```

# Model
model = DecisionTreeClassifier()

```

```

# K-Fold
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

```

```

scores = cross_val_score(
    model,
    X,
    y,
    cv=kf
)

```

```

print("Scores:", scores)
print("Mean Accuracy:", scores.mean())

```

```

Scores: [1.          0.96666667 0.93333333 0.93333333 0.93333333]
Mean Accuracy: 0.9533333333333335

```

+

✂

📄

📄

▶

■

🔄

▶▶

Code

▼

JupyterLab



Python [conda env:base] * ○

[4]: *#stratiedKFold*

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
```

```
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target
```

```
# Model
model = DecisionTreeClassifier()
```

```
# Stratified K-Fold
skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)
```

```
scores = cross_val_score(
    model,
    X,
    y,
    cv=skf
)
```

```
print("Scores:", scores)
print("Mean Accuracy:", scores.mean())
```

```
Scores: [1.          0.96666667 0.93333333 0.96666667 0.9        ]
Mean Accuracy: 0.9533333333333335
```

Hyperparameter Tuning.

Hyperparameter tuning is the process of finding the best hyperparameter values for a machine learning model to improve performance.

What are Hyperparameter Tuning?

Hyperparameters are settings that can be chosen before training the model.

Examples

Algorithm	Hyperparameters
KNN	n_neighbors (K)
Decision Tree	max_depth, min_samples_split
Random Forest	n_estimators, max_depth
SVM	C, kernel, gamma

Example

for KNN:

- K=3
- K=5
- K=7

The value of K is hyperparameter .

Why Hyperparameter Tuning?

Different hyperparameter values can produce different results.

Example

- $k=1 \rightarrow \text{Accuracy} = 85\%$
- $k=3 \rightarrow \text{Accuracy} = 90\%$
- $k=5 \rightarrow \text{Accuracy} = 95\%$

The model performance changes when the hyperparameter changes.

Goal

Find the best combination of hyperparameters that gives the highest model performance.

Benefits.

- Improves model accuracy.
- Reduce overfitting .
- Improves generalization.
- Helps find the optimal model configuration .

Hyperparameter Tuning Methods

1. GridSearchCV

GridSearchCV this all possible combinations of hyperparameters and selects the combination with the best performance.

Example

Suppose:

- $\text{max_depth} = [3,5,7]$
- $\text{min_samples_split} = [2,4]$

GridSearchCV

All combinations are evaluated using cross validation

Advantage

- Finds the best combination from the given grid.
- Easy to understand.
- Easy to implement

Disadvantage

- slow when many parameters exist
- Computationally expensive.

2. RandomizedSearchCV

RandomizedSearchCV randomly selects a fixed number of parameters combinations instead of testing all the combinations.

Example

Suppose they are

- 100 possible outcomes

RandomizedSearchCV may test:

- 10 random combinations

Advantages

- Faster than GridSearchCV
- Works well with large parameter spaces

- Lower computational cost.

Disadvantages

- May miss the absolute best combination

3. Optuna Bsics

Optuna is a modern hyperparameter optimization library.

Instead of checking all the combinations or random combinations, it learns from the previous trials and intelligently chooses better hyperparameter.

How optuna works ?

```
Trial 1
  ↓
Evaluate Model
  ↓
Learn Results
  ↓
Suggest Better Parameters
  ↓
Trial 2
```

this process continuous until the best parameter found.

Adavntages

- Fater optimization.
- Smater search strategy.
- Can stop poor performing trials early(pruning).

- Widely used in machine learning and deep learning.

Disadvantages

- Requires installing an additional library.
- Slightly more complex than GridSearchCV

Comparison

Method	Search Type	Speed
GridSearchCV	Tests all combinations	Slow
RandomizedSearchCV	Tests random combinations	Faster
Optuna	Learns and chooses better combinations	Fastest

```
[5]: pip install optuna
```

```
Collecting optuna
  Downloading optuna-4.9.0-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: alembic>=1.5.0 in c:\users\admin\anaconda3\lib\site-packages (from optuna) (1.13.3)
Collecting colorlog (from optuna)
  Downloading colorlog-6.10.1-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: numpy in c:\users\admin\anaconda3\lib\site-packages (from optuna) (1.26.4)
Requirement already satisfied: packaging>=20.0 in c:\users\admin\anaconda3\lib\site-packages (from optuna) (24.1)
Requirement already satisfied: sqlalchemy>=1.4.2 in c:\users\admin\anaconda3\lib\site-packages (from optuna) (2.0.34)
Requirement already satisfied: tqdm in c:\users\admin\anaconda3\lib\site-packages (from optuna) (4.66.5)
Requirement already satisfied: PyYAML in c:\users\admin\anaconda3\lib\site-packages (from optuna) (6.0.1)
Requirement already satisfied: Mako in c:\users\admin\anaconda3\lib\site-packages (from alembic>=1.5.0->optuna) (1.2.3)
Requirement already satisfied: typing-extensions>=4 in c:\users\admin\anaconda3\lib\site-packages (from alembic>=1.5.0->optuna) (4.11.0)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\admin\anaconda3\lib\site-packages (from sqlalchemy>=1.4.2->optuna) (3.0.1)
Requirement already satisfied: colorama in c:\users\admin\anaconda3\lib\site-packages (from colorlog->optuna) (0.4.6)
```

```

[9]: ##Grid
      from sklearn.datasets import load_iris
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.model_selection import GridSearchCV

      #datasets
      iris=load_iris()
      X=iris.data
      y=iris.target

      #model
      model=DecisionTreeClassifier()

      #Parameter grid
      param_grid={
          'max_depth':[2,3,4,5],
          'min_samples_split':[2,4,6]}

      #Grid Search
      grid=GridSearchCV(
          model,
          param_grid,
          cv=5,
          scoring='accuracy'
      )
      grid.fit(X,y)
      print("Best Parameters:",grid.best_params_)
      print("Best Score:",grid.best_score_)
    
```

```

Best Parameters: {'max_depth': 3, 'min_samples_split': 2}
Best Score: 0.9733333333333334
    
```

```

[12]: ##Grid
      from sklearn.datasets import load_iris
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.model_selection import RandomizedSearchCV

      #datasets
      iris=load_iris()
      X=iris.data
      y=iris.target

      #model
      model=DecisionTreeClassifier()

      #Parameter grid
      param_dist={
          'max_depth':[2,3,4,5,7,8],
          'min_samples_split':[2,4,6,8,10]}

      #Grid Search
      random_search=RandomizedSearchCV(
          model,
          param_distributions=param_dist,
          n_iter=10,
          cv=5,
          random_state=42
      )
      random_search.fit(X,y)
      print("Best Parameters:",random_search.best_params_)
      print("Best Score:",random_search.best_score_)
  
```

```

Best Parameters: {'min_samples_split': 8, 'max_depth': 3}
Best Score: 0.9733333333333334
  
```

[13]: !pip install optuna

```
Requirement already satisfied: optuna in c:\users\admin\anaconda3\lib\site-packages (4.9.0)
Requirement already satisfied: alembic>=1.5.0 in c:\users\admin\anaconda3\lib\site-packages (from optuna) (1.13.3)
Requirement already satisfied: colorlog in c:\users\admin\anaconda3\lib\site-packages (from optuna) (6.10.1)
Requirement already satisfied: numpy in c:\users\admin\anaconda3\lib\site-packages (from optuna) (1.26.4)
Requirement already satisfied: packaging>=20.0 in c:\users\admin\anaconda3\lib\site-packages (from optuna) (24.1)
Requirement already satisfied: sqlalchemy>=1.4.2 in c:\users\admin\anaconda3\lib\site-packages (from optuna) (2.0.34)
Requirement already satisfied: tqdm in c:\users\admin\anaconda3\lib\site-packages (from optuna) (4.66.5)
Requirement already satisfied: PyYAML in c:\users\admin\anaconda3\lib\site-packages (from optuna) (6.0.1)
Requirement already satisfied: Mako in c:\users\admin\anaconda3\lib\site-packages (from alembic>=1.5.0->optuna) (1.2.3)
Requirement already satisfied: typing-extensions>=4 in c:\users\admin\anaconda3\lib\site-packages (from alembic>=1.5.0->optuna) (4.11.0)
Requirement already satisfied: greenlet!=0.4.17 in c:\users\admin\anaconda3\lib\site-packages (from sqlalchemy>=1.4.2->optuna) (3.0.1)
Requirement already satisfied: colorama in c:\users\admin\anaconda3\lib\site-packages (from colorlog->optuna) (0.4.6)
Requirement already satisfied: MarkupSafe>=0.9.2 in c:\users\admin\anaconda3\lib\site-packages (from Mako->alembic>=1.5.0->optuna) (2.1.3)
```

```
[16]: import optuna
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

#datasets
iris=load_iris()
X=iris.data
y=iris.target

#objective
def objective(trial):
    max_depth=trial.suggest_int(
        "max_depth",
        2,
        10
    )
```


File Edit View Code

JupyterLab Python [conda env:base] *

```
min_samples_split=trial.suggest_int(
    "min_samples_split",
    2,
    10
)

model=DecisionTreeClassifier(
    max_depth=max_depth,
    min_samples_split=min_samples_split
)

score=cross_val_score(
    model,
    X,
    y,
    cv=5
).mean()
return score

#Study
study=optuna.create_study(
    direction="maximize"
)

study.optimize(
    objective,
    n_trials=20
)

print("Best parameters:")
print(study.best_params)

print("Best score:")
print(study.best_value)
```

```

[I 2026-06-12 12:35:09,404] A new study created in memory with name: no-name-bdf7ca21-39ea-42d4-ac8e-b0bc726a66c1
[I 2026-06-12 12:35:09,421] Trial 0 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_s
plit': 7}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,434] Trial 1 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_s
plit': 6}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,451] Trial 2 finished with value: 0.9533333333333334 and parameters: {'max_depth': 7, 'min_samples_s
plit': 3}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,474] Trial 3 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_s
plit': 3}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,501] Trial 4 finished with value: 0.9600000000000002 and parameters: {'max_depth': 7, 'min_samples_s
plit': 2}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,537] Trial 5 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_s
plit': 6}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,567] Trial 6 finished with value: 0.9600000000000002 and parameters: {'max_depth': 4, 'min_samples_s
plit': 4}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,607] Trial 7 finished with value: 0.9666666666666668 and parameters: {'max_depth': 8, 'min_samples_s
plit': 2}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,632] Trial 8 finished with value: 0.9666666666666668 and parameters: {'max_depth': 9, 'min_samples_s
plit': 9}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,652] Trial 9 finished with value: 0.9666666666666668 and parameters: {'max_depth': 9, 'min_samples_s
plit': 4}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,697] Trial 10 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_s
plit': 10}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,735] Trial 11 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_s
plit': 7}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,767] Trial 12 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_s
plit': 7}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,802] Trial 13 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_s
plit': 6}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,850] Trial 14 finished with value: 0.9733333333333334 and parameters: {'max_depth': 3, 'min_samples_s
plit': 8}. Best is trial 14 with value: 0.9733333333333334.
[I 2026-06-12 12:35:09,895] Trial 15 finished with value: 0.9733333333333334 and parameters: {'max_depth': 3, 'min_samples_s
plit': 8}. Best is trial 14 with value: 0.9733333333333334.
[I 2026-06-12 12:35:09,935] Trial 16 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_s
plit': 9}. Best is trial 14 with value: 0.9733333333333334.
[I 2026-06-12 12:35:09,968] Trial 17 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 8}. Be

```

```

plit': 4}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,697] Trial 10 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_
split': 10}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,735] Trial 11 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_
split': 7}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,767] Trial 12 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_
split': 7}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,802] Trial 13 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_
split': 6}. Best is trial 0 with value: 0.9666666666666668.
[I 2026-06-12 12:35:09,850] Trial 14 finished with value: 0.9733333333333334 and parameters: {'max_depth': 3, 'min_samples_
split': 8}. Best is trial 14 with value: 0.9733333333333334.
[I 2026-06-12 12:35:09,895] Trial 15 finished with value: 0.9733333333333334 and parameters: {'max_depth': 3, 'min_samples_
split': 8}. Best is trial 14 with value: 0.9733333333333334.
[I 2026-06-12 12:35:09,935] Trial 16 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_
split': 9}. Best is trial 14 with value: 0.9733333333333334.
[I 2026-06-12 12:35:09,968] Trial 17 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 8}. Be
st is trial 14 with value: 0.9733333333333334.
[I 2026-06-12 12:35:10,004] Trial 18 finished with value: 0.9666666666666668 and parameters: {'max_depth': 6, 'min_samples_
split': 10}. Best is trial 14 with value: 0.9733333333333334.
[I 2026-06-12 12:35:10,035] Trial 19 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 8}. Be
st is trial 14 with value: 0.9733333333333334.
Best parameters:
{'max_depth': 3, 'min_samples_split': 8}
Best score:
0.9733333333333334

```

[]: